

Manual De Sistema De Información

Integrantes: Benjamin Tiritera, Benjamin Montero, Lautaro Alba

Materia: Sistema de Información

Proyecto: Formulario HTML conectado a una Base de Datos SQLite

PARTE 1: MANUAL DE USUARIO

1_

A_ ¿Para qué sirve esta página?

Esta aplicación web te permite ingresar tres datos principales:

- Nombre (Texto)
- Valor (Número/Monto)
- Fecha (Día, mes y año)

Al tocar el botón de guardado, la información no se pierde de vista, sino que se envía a una base de datos SQLite y aparece guardada abajo en una lista en tiempo real.

B_ Pasos para usar el formulario

1. Abrir la aplicación: Abre el archivo index.html haciéndole doble clic desde cualquier navegador web como xr ejemplo Google
2. Llenar los campos:
 - En la casilla Nombre, escribe el nombre de la persona, producto o registro.
 - En la casilla Valor, ingresa un número o precio donde se pueden us.
 - En la casilla Fecha, selecciona la fecha usando el calendario desplegable.
3. Guardar en la Base de Datos:
 - Haz clic en el botón azul en donde dice Guardar el SQL.
 - Si dejaste algún campo vacío, el navegador te avisará que debes completarlo.
4. Ver los registros guardados:
 - Apenas hagas clic en guardar, los campos se van a limpiar solos para que puedas cargar otro registro.
 - Abajo, en la sección "Datos guardados en la BD", verás cómo aparece tu nuevo registro en una lista ordenada.

PARTE 2: MANUAL TÉCNICO

2_

A_ Explicación de la Arquitectura y Conexión

¿Cómo se conecta HTML con SQLite?

El código HTML por sí solo solo sirve para mostrar botones, textos y casilleros en la pantalla; no tiene la capacidad de guardar información en una base de datos por su cuenta.

Para conectar el HTML a la base de datos sin usar servidores complicados, usamos JavaScript junto con una librería llamada sql.js.

El circuito de la información funciona así:

1. Captura: El usuario escribe en los inputs del HTML y presiona el botón.
2. Intercepción: JavaScript frena la recarga de la página (e.preventDefault()) y lee el contenido escrito.
3. Ejecución SQL: JavaScript le pasa los datos a la librería sql.js, la cual corre el motor de SQLite y ejecuta las instrucciones SQL (INSERT INTO).
4. Muestra: JavaScript vuelve a hacer una consulta SQL (SELECT) a la base de datos y dibuja los resultados en la lista del HTML.

B_ Código Fuente Completo (index.html)

inicio del html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Formulario SQLite</title>
  <!-- Librería para usar SQLite en el navegador -->
  <script src="https://cdn.jsdelivr.net/npm/sql.js/dist/sql-wasm.js"></script>
</head>
<body>

  <h2>Registrar Datos</h2>

  <!-- Formulario principal -->
  <form id="miFormulario">
    <label>Nombre:</label><br>
    <input type="text" id="nombre" required><br><br>

    <label>Valor:</label><br>
    <input type="number" step="0.01" id="valor" required><br><br>

    <label>Fecha:</label><br>
    <input type="date" id="fecha" required><br><br>

    <button type="submit">Guardar en SQL</button>
  </form>

  <h3>Datos guardados en la BD</h3>
  <ul id="listaRegistros"></ul>

  <!-- Lógica y conexión a SQLite -->
```

```
<script>
let db;
```

Aca iniciamos el SQLite:

```
initSqlJs({ locateFile: file => `https://cdnjs.cloudflare.com/ajax/libs/sql.js/1.8.0/${file}`
}).then(SQL => {
  db = new SQL.Database();
```

Creamos una tabla si no existia

```
db.run("CREATE TABLE IF NOT EXISTS datos (id INTEGER PRIMARY KEY
AUTOINCREMENT, nombre TEXT, valor REAL, fecha TEXT);");
});
```

Aca es el evento del formulario

```
document.getElementById('miFormulario').addEventListener('submit', function(e) {
  e.preventDefault();

  const nombre = document.getElementById('nombre').value;
  const valor = parseFloat(document.getElementById('valor').value);
  const fecha = document.getElementById('fecha').value;
```

Aca ponemos los datos en SQLite

```
const stmt = db.prepare("INSERT INTO datos (nombre, valor, fecha) VALUES (?, ?, ?)");
stmt.run([nombre, valor, fecha]);
stmt.free();

this.reset();
mostrarRegistros();
});
```

Aca el html consulta y muestra el SQLite

```
function mostrarRegistros() {
  const lista = document.getElementById('listaRegistros');
  lista.innerHTML = "";

  const res = db.exec("SELECT nombre, valor, fecha FROM datos");
  if (res.length > 0) {
    const filas = res[0].values;
    filas.forEach(row => {
      const li = document.createElement('li');
      li.textContent = `Nombre: ${row[0]} | Valor: ${row[1]} | Fecha: ${row[2]}`;
      lista.appendChild(li);
```

```
    });  
  }  
}  
</script>  
  
</body>  
</html>
```

C_ Explicación del Código Paso a Paso

1. La Estructura HTML

1. `<script src="../../../sql-wasm.js">`: Carga la librería sql.js en el navegador para habilitar las funciones de SQLite.
2. `<form id="miFormulario">`: Agrupa los casilleros de entrada y les da una identidad (id) para capturarlos con JavaScript.
3. `<input id="nombre">`, `<input id="valor">`, `<input id="fecha">`: Son los tres campos requeridos para la toma de datos.
4. `<ul id="listaRegistros">`: Es la lista desordenada vacía donde más tarde se van a ir agregando los elementos que leamos de la base de datos.

2. Inicialización de la Base de Datos pero con JavaScript

1. `initSqlJs(...)`: Prepara la librería en el navegador.
2. `db = new SQL.Database()`: Crea la base de datos en la memoria del navegador.
3. `db.run("CREATE TABLE IF NOT EXISTS datos...")`: Ejecuta la orden SQL para crear la tabla datos con cuatro columnas: id, nombre, valor y fecha.

3. Captura y Guardado de Datos

1. `e.preventDefault()`: Detiene la recarga automática de la página al apretar el botón de enviar.
2. `document.getElementById(...).value`: Extrae el texto o número que el usuario escribió en cada casillero.
3. `db.prepare("INSERT INTO datos (nombre, valor, fecha) VALUES (?, ?, ?)")`: Prepara la orden SQL para insertar los valores de forma segura dentro de la tabla.
4. `stmt.run([nombre, valor, fecha])`: Ejecuta la orden guardando los datos reales dentro de SQLite.

4. Lectura y Muestra en Pantalla

1. `db.exec("SELECT nombre, valor, fecha FROM datos")`: Ejecuta la consulta SQL que trae todos los registros que existen en la tabla.
2. `filas.forEach(row => { ... })`: Recorre los registros devueltos por SQLite, crea un elemento `<li>` por cada fila y lo engancha a la lista principal del HTML.